

****The Single Source of Truth for the Entire Platform**** > Last Updated: 12 September 2026 · Version 3.0.0 > Author: Somnath Dey · Contact: somnathdey269@gmail.com --- ## 📄 Table of Contents | # | Module | Status | |---|-----|-----| | [1](#1--project-overview--architecture) | Project Overview & Architecture | ✅ LIVE | | [2](#2--dhan-api-integration-broker-layer) | Dhan API Integration (Broker Layer) | ✅ LIVE | | [3](#3--ai-recommendation-engine) | AI Recommendation Engine | ✅ LIVE | | [4](#4--stocks-universe-module) | Stocks Universe Module | ✅ LIVE | | [5](#5--financial-screener--open-screener-studio) | Financial Screener & Open Screener Studio | ✅ LIVE | | [6](#6--company-financials--ground-truth) | Company Financials & Ground Truth | ✅ LIVE | | [7](#7--my-portfolio-module) | My Portfolio Module | ✅ LIVE | | [8](#8--my-watchlist--custom-portfolios) | My Watchlist / Custom Portfolios | ✅ LIVE | | [9](#9--data-vault-60-day-tick-vault--quant-matrix) | Data Vault (60-Day Tick Vault & Quant Matrix) | ✅ LIVE | | [10](#10--market-trends-module) | Market Trends Module | ✅ LIVE | | [11](#11--dalal-street-news-live-financial-intelligence) | Dalal Street News (Live Financial Intelligence) | ✅ LIVE | | [12](#12--chart--rules-studio-dynamic-trigger-studio) | Chart & Rules Studio (Dynamic Trigger Studio) | ✅ LIVE | | [13](#13--bot-studio-ai-co-pilot--strategy-builder) | Bot Studio (AI Co-Pilot & Strategy Builder) | ✅ LIVE | | [14](#14--xkiro-ai-gateway-integration) | xKiro AI Gateway Integration | ✅ LIVE | | [15](#15--corporate-filings-bse-sebi-lodr) | Corporate Filings (BSE SEBI LODR) | ✅ LIVE | | [16](#16--authentication--user-management) | Authentication & User Management | ✅ LIVE | | [17](#17--admin-portal-port-3001) | Admin Portal (Port 3001) | ✅ LIVE | | [18](#18--settings--configuration) | Settings & Configuration | ✅ LIVE | | [19](#19--websocket-real-time-system) | WebSocket Real-Time System | ✅ LIVE | | [20](#20--frontend-architecture) | Frontend Architecture | ✅ LIVE | | [21](#21--database-architecture) | Database Architecture | ✅ LIVE | | [22](#22--startup--background-services) | Startup & Background Services | ✅ LIVE | | [23](#23--complete-api-reference) | Complete API Reference | ✅ LIVE | | [24](#24--development-log) | Development Log | 📅 LIVING | | [25](#25--discussed-but-not-yet-built) | Discussed But Not Yet Built | 💡 BACKLOG | | [26](#26--roadmap--backlog) | Roadmap & Backlog | 🗓️ PLANNED | | [27](#27--known-issues--gotchas) | Known Issues & Gotchas | 🐞 WATCH | | [28](#28--design-decisions) | Design Decisions | 📁 ARCHIVED | --- # 1. 📄 Project Overview & Architecture ## What It Does (Plain English) This is a ****full-stack Indian stock market terminal**** that lets you: - See live prices of ****5,092+ BSE & NSE stocks**** in real-time - Get ****AI-powered stock recommendations**** across Intraday, Swing, and Wealth horizons - ****Place real buy/sell orders**** through your Dhan broker account - ****Screen stocks**** using financial formulas (like "PE < 25 AND ROCE > 15") - Build and backtest ****automated trading bots**** - Track ****live news****, ****market trends****, and ****corporate filings**** - Store ****60 days of 1-minute historical candles**** (~17 GB) for quant analysis ## Three-App Architecture

APEX EQUITIES PLATFORM		
🖥️ FRONTEND	⚙️ BACKEND (API)	🔧 ADMIN PORTAL
Next.js 16	FastAPI (Python)	Next.js (Separate App)
Port: 3000	Port: 8000	Port: 3001
React 19	SQLite DBs	Single-user auth
TailwindCSS 4	WebSocket /ws/	Strategy management
TypeScript 5	Background workers	Universe Inspector
Lucide Icons	DhanHQ SDK	Quant Copilot

Directory Structure

```

Stock/
├── frontend/                                # User-facing Next.js dashboard (Port 3000)
│   └── src/
│       ├── app/                            # Next.js App Router pages (9 routes)
│       │   ├── page.tsx                    # Root → redirects to main app
│       │   ├── chart/                      # Chart & Rules Studio page
│       │   ├── news/                      # Dalal Street News page
│       │   ├── portfolio/                 # My Portfolio page
│       │   ├── recommendations/          # AI Recommendations page
│       │   ├── settings/                 # Settings page
│       │   ├── stocks/                   # Stocks Universe page
│       │   ├── trends/                   # Market Trends page
│       │   ├── vault/                    # Data Vault page
│       │   └── watchlist/                 # My Watchlist page
│       ├── components/                   # 26 React components (TSX)
│       ├── services/                     # api.ts – 733 lines, ALL backend calls
│       ├── types/                        # index.ts – 853 lines, TypeScript interfaces
│       └── constants/                    # presetStrategies.ts, stockColumnCatalog.ts, xkiroModels.ts
├── backend/                                # FastAPI Python backend (Port 8000)
│   ├── app/
│   │   ├── main.py                        # FastAPI app entry, CORS, WebSocket, startup hooks
│   │   ├── core/
│   │   │   └── config.py                 # Settings (strategy defaults, F&O stock universe)
│   │   ├── api/v1/                       # 13 REST API route files
│   │   │   ├── health.py                 # Health check
│   │   │   ├── universe.py               # Stock universe, charts, indices, trends
│   │   │   ├── recommendations.py        # AI recommendation endpoints
│   │   │   ├── admin_portal.py           # Admin intelligence portal
│   │   │   ├── trade.py                  # Dhan live trading & orders
│   │   │   ├── portfolios.py             # Screener & custom portfolios
│   │   │   ├── news.py                   # Live news & market wire
│   │   │   ├── bots.py                   # Dynamic Bot Studio
│   │   │   ├── rules.py                  # Dynamic rules & triggers
│   │   │   ├── admin.py                  # Dhan/xKiro/Cost settings
│   │   │   ├── auth.py                   # Authentication & alerts
│   │   │   ├── corporate_filings.py      # Corporate filings & results
│   │   │   └── historical_data_api.py    # Historical data & quant matrix
│   │   ├── engine/                       # 43 engine files (business logic core)
│   │   └── models/
│   │       └── schemas.py                 # Pydantic request/response schemas
│   ├── requirements.txt                   # Python dependencies
│   └── *.db                              # 7 SQLite databases
├── admin-portal/                          # Admin-only Next.js app (Port 3001)
│   └── src/app/
│       ├── page.tsx                      # Main admin dashboard (~113KB)
│       └── QuantCopilotView.tsx          # Quant Copilot view (~26KB)

```

Tech Stack Summary | Layer | Technology | Version | Purpose | |-----|-----|-----|-----| | ****Frontend Framework**** |
Next.js | 16.3.4 | React server/client rendering, App Router | | ****UI Library**** | React | 19.2.8 | Component-based UI | | ****Styling**** |
TailwindCSS | 4.x | Utility-first CSS | | ****Icons**** | Lucide React | 1.39.0 | Icon library | | ****Language**** | TypeScript | 5.x | Type-safe
frontend code | | ****Backend Framework**** | FastAPI | ≥0.110.0 | Async Python REST API | | ****ASGI Server**** | Uvicorn | ≥0.28.0 | FastAPI
server | | ****Validation**** | Pydantic | ≥2.6.0 | Request/Response schemas | | ****Broker SDK**** | DhanHQ | ≥2.2.0 | Live market data +
trading | | ****AI Gateway**** | xKiro / Anthropic | ≥1.4.0 | AI-powered bot conversations | | ****HTTP Client**** | HTTPX | ≥0.27.0 | Async HTTP
calls | | ****Market Data**** | yFinance | latest | Fallback price + financial data | | ****Database**** | SQLite | built-in | 7 databases for
persistence | | ****Testing**** | Pytest | ≥8.0.0 | Unit & integration tests | --- # 2. 🏠 Dhan API Integration (Broker Layer) ## What It Does
(Plain English) Dhan is our ****primary stock broker API****. Think of it as the direct phone line to the Indian stock exchanges (BSE &
NSE). Through Dhan, we: - Get ****live stock prices**** updated every second - ****Place real buy/sell orders**** on BSE & NSE - Fetch your
****portfolio positions****, ****order history****, and ****demat holdings**** - Download ****60 days of 1-minute historical candle data**** for every
stock - Get ****live market indices**** (Nifty 50, Sensex, Bank Nifty, etc.) --- ### 2.1 `dhan\_provider.py` — Core Market Data Provider >
****File****: `backend/app/engine/dhan\_provider.py` · ****Lines****: 2,411 · ****Size****: ~117 KB > This is the ****largest single file**** in the entire
project. ##### What It Does This is the ****heart of all market data****. Every stock price, every chart, every index value flows through this
file. It manages: 1. ****Live WebSocket connection**** to DhanHQ for real-time tick data 2. ****In-memory stock cache**** (`stocks\_cache`)
holding all 5,092+ equities 3. ****OHLCV chart data**** computation from historical candles 4. ****Technical indicators**** (RSI, EMA
20/50/200, VWAP, Supertrend, Bollinger Bands) 5. ****Market indices**** (Nifty 50, Sensex, Bank Nifty, Nifty Midcap, etc.) 6. ****Symbol ↔
Security ID mapping**** for Dhan API compatibility ##### Key Data Structures ****KNOWN_DHAN_SCRIP_IDS**** — Hardcoded Security
ID Lookup Table | Symbol | NSE Equity ID | BSE Equity ID | Futures ID | Lot Size | |-----|:-----|:-----|:-----|:-----|:-----| |
RELIANCE | 2885 | 500325 | 68648 | 250 | | TCS | 11536 | 532540 | 68673 | 175 | | HDFCBANK | 1333 | 500180 | 68589 | 550 | | INFY |
1594 | 500209 | 68601 | 400 | | ICICIBANK | 4963 | 532174 | 68595 | 700 | | SBIN | 3045 | 500112 | 68656 | 1500 | | BHARTIARTL |
10604 | 532454 | 68550 | 475 | | TATASTEEL | 3499 | 500470 | 68671 | 5500 | | TATAMOTORS | 3456 | 500570 | 68669 | 1400 | |
MARUTI | 10999 | 532500 | 68625 | 100 | ****`stocks\_cache`**** — In-Memory Stock Data (Per Stock) Every stock in the system holds
these fields in memory: | Field | Type | Example | Used In | |-----|-----|-----|-----| | `symbol` | str | `RELIANCE` | |
Everywhere | | `name` | str | `Reliance Industries Ltd` | | Universe table, Recommendations | | `exchange` | str | `NSE` | | Filter,
display | | `exchanges` | list | `["NSE", "BSE"]` | | Dual-listing badge | | `is\_dual\_listed` | bool | `true` | | Arbitrage detection | | `sector` |
str | `Oil, Gas & Petrochemicals` | | Sector filter, News matching | | `mcap\_category` | str | `Large Cap` | | Market cap filter | | `ltp` |
float | `1308.20` | | Main price column | | `prev\_close` | float | `1295.50` | | Change calculation | | `change` | float | `+12.70` | | Change
display | | `change\_pct` | float | `+0.98` | | Colored badge (green/red) | | `day\_high` | float | `1315.00` | | Day range display | | `day\_low` |
float | `1290.10` | | Day range display | | `high\_52w` | float | `1608.80` | | 52W range display | | `low\_52w` | float | `1120.00` | | 52W
range display | | `volume` | int | `18500000` | | Volume column, liquidity check | | `vwap` | float | `1303.44` | | VWAP indicator | |
`sparkline` | list | `[1295, 1298, 1302, ...]` | | Mini sparkline chart | | `nse\_ltp` | float | `1308.20` | | NSE price (arbitrage) | | `bse\_ltp` |
float | `1307.85` | | BSE price (arbitrage) | | `price\_diff` | float | `0.35` | | Price gap amount | | `price\_diff\_pct` | float | `0.027` | | Price gap
% | | `buy\_exchange` | str | `BSE` | | Cheapest exchange | | `sell\_exchange` | str | `NSE` | | Most expensive exchange | | `series` | str |
`EQ` | | Series classification | | `is\_etf` | bool | `false` | | ETF filter | | `instrument\_type` | str | `EQUITY` | | Instrument filter | |
`updated\_at` | float | `1726142400.5` | | Freshness check | ##### DhanHQ MarketFeed Patching The file starts by ****patching**** the
DhanHQ SDK to fix compatibility issues:

🐞 **Problem:** DhanHQ's MarketFeed WebSocket has a `\_is\_ws\_closed()` method
that crashes on certain Python versions

🔧 **Fix:** We replace it with `safe\_is\_ws\_closed()` that gracefully handles
all edge cases (None checks, state string matching)

🐞 **Problem:** DhanHQ's DhanHTTP `\_parse\_response()` swallows error details

🔧 **Fix:** We override it to extract and surface Dhan's actual error
messages from the response body

Where `dhan\_provider` Is Used | Module | How It Uses dhan_provider | |-----|-----|-----|-----|-----| | **Stocks** Universe | `get\_stocks()` → paginated stock list with filters | | **Market Indices** | `get\_market\_ticker()` → Nifty, Sensex, Bank Nifty values | | **Charts** | `get\_chart\_data()` → OHLCV candles + technical indicators | | **Recommendations** | `stocks\_cache` → live LTP for entry/exit monitoring | | **Trends** | `stocks\_cache` → breadth, momentum, sector calculations | | **Data Vault** | Dhan historical API → 60-day 1-min candle downloads | | **Portfolio** | Live tick updates → position P&L calculation | | **Bot Backtest** | `get\_chart\_data()` → historical simulation | | **Screener** | `stocks\_cache` → metric evaluation against all stocks | | **WebSocket** | `get\_stock\_quote\_tick()` → real-time tick broadcast | | **News** | Sector mapping → stock-to-sector matching | --- ### 2.2 `dhan\_totp\_auth.py` — Automated Daily TOTP Login > **File**: `backend/app/engine/dhan\_totp\_auth.py` · **Lines**: 290 · **Size**: ~12 KB #### What It Does (Plain English) Dhan requires you to log in every day with a new access token (like a daily password). This file **automates** that process so you never have to manually log in: 1. You save your **Client ID**, **Trading PIN**, and **TOTP Secret** once 2. Every morning at **7:30 AM IST**, a background thread automatically: - Generates a fresh TOTP code (like Google Authenticator) - Sends it to Dhan's login API - Gets a fresh 24-hour access token - Reconnects the live market data feed 3. The token is **saved to disk** at `dhan\_session.json` so it survives restarts #### Configuration Storage **File**: `dhan\_totp\_config.json`

```
{
  "client_id": "1234567890",
  "pin": "123456",
  "totp_secret": "BASE32ENCODEDSTRING"
}
```

File: `dhan\_session.json`

```
{
  "client_id": "1234567890",
  "access_token": "eyJhbG...<JWT token>",
  "generated_at": "2026-09-12 07:30:15",
  "expires_at": "2026-09-13 07:30:15"
}
```

API Endpoints for TOTP | Endpoint | Method | Input | Output | What It Does | |-----|-----|-----|-----|-----| | `/api/v1/admin/dhan/totp/status` | GET | None | `{is\_configured, client\_id, last\_renewed\_at, scheduler\_running}` | Shows TOTP setup status | | `/api/v1/admin/dhan/totp/configure` | POST | `{client\_id, pin, totp\_secret}` | `{success, message, live\_feed\_connected}` | Saves credentials + tests + connects | | `/api/v1/admin/dhan/totp/refresh-now` | POST | None | `{success, access\_token, expires\_at}` | Force-generates new token right now | --- ### 2.3 `dhan\_trade\_service.py` — Live Trading Execution > **File**: `backend/app/engine/dhan\_trade\_service.py` · **Lines**: 697 · **Size**: ~33 KB #### What It Does (Plain English) This is the **actual money execution layer**. When you click "Buy" on a recommendation, this file: 1. Sends the order to Dhan's API → gets placed on BSE/NSE exchange 2. Tracks your **open positions** with real-time P&L 3. Manages **dual stop-loss system** (SL1 = primary, SL2 = breakeven after partial profit) 4. Tracks **order history** and allows cancellation 5. Fetches your **demat holdings** (long-term stocks in your account) #### Credential Loading Priority

1. 🔍 Check `dhan_session.json` (auto-generated by TOTP scheduler)
↓ (if not found)
2. 🔍 Check `dhan_config.json` (manually saved credentials)
↓ (if not found)
3. 🔍 Check environment variables `DHAN_CLIENT_ID` + `DHAN_ACCESS_TOKEN`

Trade API Endpoints | Endpoint | Method | Input Parameters | Output Fields | What It Does | |-----|-----|-----|
|-----|-----| | `/api/v1/trade/status` | GET | None | `{is\_connected, client\_id, account\_name, cash\_balance, margin\_available}` | Check broker connectivity | | `/api/v1/trade/settings` | POST | `{client\_id?, access\_token?, auto\_sl2\_breakeven?}` | `{success, message}` | Save/update Dhan API keys | | `/api/v1/trade/order` | POST | `{symbol, quantity, price, stop\_loss\_1, stop\_loss\_2?, target\_price?, product\_type, order\_type}` | `{order\_id, status, message, exchange\_order\_id}` | Place buy order with dual stops | | `/api/v1/trade/positions` | GET | None | `{positions[], summary{open\_count, total\_unrealized\_pnl, total\_invested}}` | Get live open positions | | `/api/v1/trade/squareoff` | POST | `{position\_id, symbol?, quantity?}` | `{success, message, exit\_price}` | Sell/close a position | | `/api/v1/trade/orders` | GET | None | `{orders[], total\_count}` | All orders placed today | | `/api/v1/trade/order/cancel` | POST | `{order\_id}` | `{success, message}` | Cancel a pending order | | `/api/v1/trade/holdings` | GET | None | `{holdings[], summary{total\_invested, total\_current\_value, total\_pnl, total\_day\_pnl}}` | Demat portfolio holdings | #### Order SL Tracker **File**: `backend/app/engine/order\_sl\_tracker.json` Tracks which orders have active stop-losses and their states:

```
{
  "order_123": {
    "symbol": "RELIANCE",
    "buy_price": 1305.00,
    "sl1_price": 1285.00,
    "sl2_price": 1305.00,
    "sl2_status": "WAITING",
    "target_price": 1380.00
  }
}
```

Frontend Display **Component**: `PortfolioView.tsx` (45 KB) + `OrderPlacementModal.tsx` (35 KB) The order modal shows: - Symbol name + current LTP - Quantity input + calculated investment amount - Price input (Limit/Market) - Stop Loss 1 (mandatory) - Stop Loss 2 / Breakeven (optional, auto-moves to entry after partial profit) - Target price - Product type selector (INTRADAY / CNC) -- - # 3. 🧠 AI Recommendation Engine ## What It Does (Plain English) This is the **brain of the platform**. It automatically scans 5,092+ stocks every 30 minutes and finds the best ones to buy. Each recommendation tells you: - **What to buy** (stock name + symbol) - **At what price** (entry range) - **Where to book profit** (target price) - **Where to cut losses** (stop loss) - **Why it's a good pick** (18-parameter evidence analysis) --- ### 3.1 `recommendation\_engine.py` — Multi-Horizon AI Stock Picker > **File**: `backend/app/engine/recommendation\_engine.py` . **Lines**: 2,724 . **Size**: ~138 KB > This is the **second largest file** in the entire project. ##### The 12-Strategy Library The engine runs **12 distinct strategies** across 3 time horizons: ⚡ Intraday Strategies (Exit by 3:15 PM) * # | Strategy Name | Version | Core Logic | |---|-----|-----|-----| | 1 | Intraday VWAP & Volume Spike | v1.0-INTRA-VWAP | Enters above VWAP when 15-min volume surges >2.5x | | 2 | Opening Range Breakout (ORB) | v1.0-INTRA-ORB | Breaks first 15-minute high with index confirmation | | 3 | Intraday Pullback to EMA | v1.0-INTRA-PULLBACK | Buys dips to 20 EMA in strong intraday leaders | 📈 Short-Term Swing (1–4 Weeks) * # | Strategy Name | Version | Core Logic | |---|-----|-----|-----| | 4 | Institutional VCP Breakout | v1.0 | Volatility Contraction Pattern + delivery surge | | 5 | Smart Money Delivery Accumulation | v1.1-DELIVERY | 3 sessions of delivery >55% with supply dry-up | | 6 | 52-Week High Stage-2 Momentum | v1.0-52W | Fresh 52W highs in Large & Mid caps | | 7 | 20 EMA Trend Pullback | v1.0-EMA | Retest of breakout near 20 EMA on declining volume | | 8 | Sector Relative Strength Leader | v2.0-SECTOR | Top stocks in sectors beating Sensex by >2.5% | | 9 | Post-Earnings Drift (PEAD) | v1.0-PEAD | Quarterly profit >25% YoY with gap continuation | 🏠 Long-Term Wealth (3–12 Months) * # | Strategy Name | Version | Core Logic | |---|-----|-----|-----| | 10 | High RoCE Zero-Debt Compounder | v1.0-WEALTH-ROCE | RoCE >22%, D/E <0.2, 18% 3Y profit CAGR | | 11 | Growth at Reasonable Price (GARP) | v1.0-WEALTH-GARP | PEG <1.0, expanding EBITDA margins | | 12 | Valuation Floor Reversal | v1.0-WEALTH-REVERSAL | Blue-chips at 3Y valuation floors | #### Apex 18-Parameter Engine (v1.2 — Active Champion) Every recommendation is scored against **18 parameters**. A stock needs a minimum **10 of 18** applicable parameters to qualify and must score **96+/100**. * # | Parameter | What It Checks (Plain English) | |---|-----|-----|-----| | 1 | Financial Stability Gate | Company is not making losses, has positive net worth | | 2 | Revenue Growth | Sales are growing, not shrinking | | 3 | Profit Growth | Net profit is increasing year-over-year | | 4 | Return on Capital (RoCE) | Management generates good returns on money invested | | 5 | Debt Safety | Company is not drowning in debt | | 6 | Promoter Pledge Check | Promoters haven't pledged shares (bankruptcy risk) | | 7 | Chart Base Structure | Price has built a proper

technical base pattern || 8 | Volume Confirmation | Buying volume supports the price move || 9 | Delivery % | Real shares changing hands (not just traders) || 10 | Sector Strength | Stock's sector is outperforming the market || 11 | Market Regime | Overall market is not in panic mode || 12 | Risk-Reward Ratio | Potential profit is $\geq 2x$ the stop-loss risk || 13 | Liquidity Floor | Stock has enough trading volume for easy entry/exit || 14 | 52W High Proximity | Stock is close to 52-week highs (momentum) || 15 | EMA Alignment | Moving averages are properly aligned (bullish) || 16 | VIX Safety | Market volatility is within acceptable range || 17 | ASM/GSM Check | Stock is not under exchange surveillance || 18 | Earnings Blackout | No quarterly result date within 7 days | ##### Daily Caps & Deduplication - **Max 2** Swing recommendations per day - **Max 1** Growth recommendation per day - **Dedup lock scope**:

ENTIRE_DAY_ALL_HORIZONS (same stock can't be recommended twice in any horizon) ##### API Endpoints | Endpoint | Method | Input | Output | Frontend Component | |-----|-----|-----|-----|-----| | /api/v1/recommendations/metrics | GET | None | {total_picks, success_count, win_rate_pct, stopped_out, top_strategy} | Performance scorecard at top ||

/api/v1/recommendations/active | GET | ?horizon=&date= | {count, recommendations} | Main recommendation cards ||

/api/v1/recommendations/sessions | GET | None | {sessions[{date, count}]} | Session date picker ||

/api/v1/recommendations/history | GET | ?limit=50&date= | {count, history[]} | Historical tab ||

/api/v1/recommendations/{id}/reason | GET | rec_id | {reasons[], evidence{}, entry_range, target, stop_loss} | "Why Buy" evidence drawer ||

/api/v1/recommendations/intraday/active | GET | None | {recommendations[]} | Intraday tab filter ||

/api/v1/recommendations/intraday/trimmed | GET | None | {recommendations[]} | Trimmed trades tab ||

/api/v1/recommendations/scan | POST | ?force=true | {scanned, published, skipped} | Manual scan trigger ||

/api/v1/recommendations/reset | POST | None | {message, scan_result} | Wipe & regenerate all | ##### Recommendation Data Fields (Per Recommendation) | Field | Type | Example | Displayed Where | |-----|-----|-----|-----| | id | str | "rec_abc12345" | Internal reference ||

symbol | str | "RELIANCE" | Card header ||

company_name | str | "Reliance Industries Ltd" | Card header ||

sector | str | "Oil, Gas & Petrochemicals" | Sector badge ||

market_cap_category | str | "Large Cap" | Cap badge ||

market_cap_cr | float | 1750000.0 | Cap display ||

bse_price | float | 1305.50 | Price reference ||

nse_price | float | 1306.10 | Price reference ||

entry_min | float | 1295.00 | "Buy ₹1,295 – ₹1,310" range ||

entry_max | float | 1310.00 | "Buy ₹1,295 – ₹1,310" range ||

target_price | float | 1420.00 | Target badge (+8.4%) ||

stop_loss | float | 1260.00 | Stop loss badge (-3.5%) ||

invalidation_price | float | 1240.00 | Hard invalidation level ||

expected_horizon | str | "2 to 4 weeks" | Horizon label ||

risk_reward_ratio | float | 2.4 | R:R display "1:2.4" ||

opportunity_score | int | 97 | Score badge ||

status | str | "WAITING_FOR_ENTRY" | Pulsing status indicator ||

status_label | str | "In Buy Range" | Human-readable status ||

reasons_json | JSON | [{"Strong delivery surge", ...}] | Evidence drawer ||

evidence_json | JSON | {chart_structure: {...}} | Evidence drawer ||

strategy_version | str | "strat_apex_v1_2" | Strategy attribution ||

created_at_str | str | "12.09.2026 10:30" | Timestamp display | ##### Storage: recommendations.db See [Section 21](#21--database-architecture) for complete schema.

Frontend Component **RecommendationDashboardView.tsx** — **171 KB** (the largest frontend component) Displays: -  **Performance scorecard** at the top (Total Picks, Win Rate, Failures, Top Strategy) -  **Horizon filter tabs** (All / Intraday / 1-Week Swing / 3-Month Growth) -  **Session date navigator** (view any past day's recommendations) -  **Recommendation cards** with entry range, target, SL, status indicator -  **Evidence drawer** (click any card to see 18-parameter breakdown) -  **History tab** (completed/closed trades with returns) --- # 4.  Stocks Universe Module ## What It Does (Plain English) This is the **main stock screener page** where you can browse, search, filter, and sort all 5,092+ BSE and NSE equities in real-time. Think of it like a supercharged version of Screener.in or Moneycontrol, but with live prices. --- ### 4.1 Data Pipeline

```

 BSE/NSE Master CSVs (bse_nse_master.csv, nse_equity_master.csv)
  ↓ loads at startup
 real_exchange_provider.py → merges + deduplicates by company name
  ↓ builds unified stocks_cache
 DhanHQ MarketFeed WebSocket → streams live ticks
  ↓ updates stocks_cache in real-time
 universe.py API → paginated, filtered, sorted response
  ↓ enriched with financial_registry metrics
 UniverseView.tsx → renders table with live prices

```

4.2 API Endpoint Deep-Dive ##### GET /api/v1/universe/stocks | Input Parameter | Type | Default | Description | |-----|-----|-----|-----| | search | string | "" | Search by symbol, company name, or sector ||

symbols | string | null |

Comma-separated symbols to filter (e.g., `"RELIANCE,TCS,INFY"`) || `sector` | string | `"ALL"` | Sector filter from 31 sectors || `exchange` | string | `"ALL"` | `NSE`, `BSE`, or `ALL` || `instrument` | string | `"ALL"` | `EQUITY`, `ETF`, `SME`, `BE`, etc. || `mcap` | string | `"ALL"` | `Large Cap`, `Mid Cap`, `Small Cap` || `sort\_by` | string | `"volume"` | Sort field: `volume`, `ltp`, `change\_pct`, `symbol`, or any metric || `sort\_dir` | string | `"desc"` | `asc` or `desc` || `page` | int | `1` | Page number (1-indexed) || `page\_size` | int | `50` | Stocks per page || `price\_diff\_only` | bool | `false` | Only dual-listed stocks with BSE≠NSE price |

****Processing Steps:**** 1. `universe\_provider.get\_stocks()` → filters `stocks\_cache` by search/sector/exchange/mcap 2. Sorts by requested field 3. Paginates results 4. `financial\_registry.get\_stock\_metrics()` → enriches each stock with 500+ financial metrics 5. Merges live Dhan quotes (LTP, change, volume) with financial data (PE, ROCE, etc.) ****Output Response:****

```
{
  "stocks": [
    {
      "symbol": "RELIANCE",
      "name": "Reliance Industries Ltd",
      "exchange": "NSE",
      "exchanges": ["NSE", "BSE"],
      "is_dual_listed": true,
      "sector": "Oil, Gas & Petrochemicals",
      "mcap_category": "Large Cap",
      "ltp": 1308.20,
      "prev_close": 1295.50,
      "change": 12.70,
      "change_pct": 0.98,
      "day_high": 1315.00,
      "day_low": 1290.10,
      "volume": 18500000,
      "vwap": 1303.44,
      "sparkline": [1295, 1298, 1302, 1305, 1308],
      "nse_ltp": 1308.20,
      "bse_ltp": 1307.85,
      "pe": 28.5,
      "roce": 14.2,
      "roe": 12.8,
      "market_cap": 1750000,
      "... 500+ more metric fields"
    }
  ],
  "total": 5092,
  "page": 1,
  "page_size": 50,
  "total_pages": 102,
  "dhan_connected": true,
  "is_market_open": true,
  "market_status": "OPEN",
  "status_label": "Market Open"
}
```

Other Universe Endpoints | Endpoint | Method | Input | Output | |-----|-----|-----|-----|

`/api/v1/universe/stocks/{symbol}` | GET | symbol path param | Single stock full data || `/api/v1/universe/stocks/{symbol}/chart` | GET | `?timeframe=15m&bars=50` | `{candles[], technicals{rsi, ema\_20, ema\_50, ema\_200, vwap, supertrend, bollinger}}` ||

``/api/v1/universe/stocks/{symbol}/details` | GET | `?exchange=NSE` | `{profile, financial_statements, news[]}` | |`
``/api/v1/universe/indices` | GET | None | `{indices[], top_gainers[], top_losers[], market_status, is_market_open}` | |`
``/api/v1/universe/sectors` | GET | None | `{sectors[]}` — all 31 sector names | ### 4.3 Frontend Component UniverseView.tsx —`
152 KB (second largest frontend component) Features: - 🔍 Search bar (instant search by symbol/name/sector) - 🍪 Filter chips (Exchange, Sector, Market Cap, Instrument Type) - 🇮🇳 Customizable column table (choose which metrics to display) - 📈 Inline sparkline charts per stock - 🔄 Real-time price updates via WebSocket - 📊 Arbitrage view (highlight BSE≠NSE price differences) - 📄 Pagination (50 stocks per page) - 📱 Click-to-sort on any column --- # 5. 📊 Financial Screener & Open Screener Studio ## What It Does (Plain English) The screener lets you **write financial queries in plain English** to find stocks matching specific criteria. For example: - `"PE < 25 AND ROCE > 15"` → Find stocks with low PE and high returns - `"sales_growth_3y > 20 AND debt_to_equity < 0.5"` → Fast-growing, low-debt companies - `"rsi < 30 AND volume > 1000000"` → Oversold stocks with high volume --- ### 5.1 ``screener_engine.py`` — AST-Based Safe Expression Evaluator > **File**: ``backend/app/engine/screener_engine.py`` · **Lines**: 607 · **Size**: ~30 KB ##### How It Works 1. **User** types a query like `"pe < 25 AND roce > 15"` 2. **Alias resolver** converts natural language → metric keys: - `"sales growth 3 years"` → `"sales_growth_3y"` - `"return on equity"` → `"roe"` - `"price to earnings"` → `"pe"` 3. **Python AST parser** safely evaluates the expression (no `eval()` — completely safe) 4. **Iterates** all 5,092 stocks, evaluates each against the expression 5. **Returns** matching stocks with full metric data, sorted and paginated ##### Supported Operators | Operator | Example | What It Means | |-----|-----|-----|-----| | `>` | `"pe > 20"` | Greater than | | `<` | `"pe < 25"` | Less than | | `>=` | `"roce >= 15"` | Greater than or equal | | `<=` | `"debt_to_equity <= 0.5"` | Less than or equal | | `==` | `"sector == 'Banking'"` | Equals (case-insensitive for strings) | | `!=` | `"exchange != 'BSE'"` | Not equals | | `AND` | `"pe < 25 AND roce > 15"` | Both conditions must be true | | `OR` | `"rsi < 30 OR rsi > 70"` | Either condition can be true | | `in` | `"sector in ['Banking', 'IT']"` | Value is in list | | `not in` | `"sector not in ['Real Estate']"` | Value is not in list | | `+` `-` `*` `/` | `"market_cap / sales"` | Arithmetic expressions | ##### API Endpoint | Endpoint | Method | Input | Output | |-----|-----|-----|-----| | `/api/v1/screener/evaluate`` | POST | `{query, page, page_size, sort_by, sort_dir}`` | `{success, stocks[], total, execution_ms, diagnostic{}}` | | `/api/v1/screener/metrics`` | GET | None | `{count, metrics[{key, name, category, unit, timeframes}]}`` | | `/api/v1/screener/apex-strategies`` | GET | None | `{count, strategies[]}`` — 32 preset strategies | --- ### 5.2 ``financial_registry.py`` — 500+ Metric Definitions > **File**: ``backend/app/engine/financial_registry.py`` · **Lines**: 1,049 · **Size**: ~36 KB **Available Metric Categories**: | Category | Example Metrics | Timeframes Supported | |-----|-----|-----|-----| | **Growth** | `"sales_growth"`, `"profit_growth"`, `"ebitda_growth"`, `"eps_growth"` | 1Y, 3Y, 5Y | | **Profitability** | `"opm"`, `"npm"`, `"roce"`, `"roe"`, `"roic"` | Latest, 1Y, 3Y, 5Y | | **Valuation** | `"pe"`, `"pb"`, `"ev_ebitda"`, `"peg"`, `"dividend_yield"` | Latest | | **Quality** | `"debt_to_equity"`, `"interest_cover"`, `"current_ratio"`, `"cfo_to_pat"` | Latest | | **Technical** | `"rsi"`, `"ema_20"`, `"ema_50"`, `"ema_200"`, `"vwap"`, `"supertrend"` | Live | | **Shareholding** | `"promoter_holding"`, `"fii_holding"`, `"dii_holding"`, `"promoter_pledged_pct"` | Latest | | **Cash Flow** | `"cfo"`, `"free_cash_flow"`, `"capex"`, `"cash_conversion_cycle"` | Latest, 3Y, 5Y | --- ### 5.3 ``preset_strategies_library.py`` — 32 Institutional Strategies > **File**: ``backend/app/engine/preset_strategies_library.py`` · **Lines**: 536 · **Size**: ~29 KB **Core Disciplines**: | Discipline | # Strategies | Example | |-----|-----|-----| | Quality & Moats | 5 | High Capital Return Compounds | | Growth Momentum | 5 | Accelerating EPS & Sales Momentum | | Value & Contrarian | 5 | Deep Value Low PE Re-raters | | Income & Dividend | 4 | Dividend Growth Aristocrats | | Technical & Momentum | 5 | 52-Week High Breakout Leaders | | Sector Rotation | 4 | Sector Relative Strength Leaders | | Special Situations | 4 | Working Capital Efficiency | Each strategy has a pre-built screener `"query"` that users can 1-click execute. --- ### 5.4 Custom Formulas Users can **save their own formulas** as reusable metrics: | Endpoint | Method | Input | Output | |-----|-----|-----|-----| | `/api/v1/screener/custom-formulas`` | GET | None | `{formulas[]}`` | | `/api/v1/screener/custom-formulas`` | POST | `{name, expression, description?, unit?}`` | `{success, formula}`` | | `/api/v1/screener/custom-formulas/{id}`` | DELETE | `formula_id`` | `{success}`` | **Storage**: `".saved_custom_formulas.json"` --- # 6. 📄 Company Financials & Ground Truth ## What It Does (Plain English) When you click on any stock, you can see its **complete financial statements** — just like reading an annual report, but structured and easy to understand: - **Balance Sheet** (what the company owns vs. owes) - **Profit & Loss** (revenue, expenses, net profit) - **Cash Flow** (where cash is coming from and going to) - **Key Ratios** (PE, ROCE, ROE, D/E, etc.) - **Shareholding Pattern** (who owns the company — promoters, FIIs, DIIs) ### 6.1 ``company_financials.py`` > **File**: ``backend/app/engine/company_financials.py`` · **Lines**: 629 · **Size**: ~28 KB Provides Ind-AS standard financial statements with **authentic ground truth** data where available, and **sector-adjusted synthetic data** for stocks without filings. ### 6.2 ``financial_ground_truth.py`` > **File**: ``backend/app/engine/financial_ground_truth.py`` · **Lines**: 548 · **Size**: ~24 KB Fetches and persists **real, audited financial figures** using yFinance → normalizes to INR ₹ Crores → stores in SQLite `"corporate_filings.db"`. **Database Schema**: `"company_financials_master"` table with 50+ fields including: `"market_cap"`, `"sales"`, `"revenue"`, `"expenses"`, `"ebitda"`, `"operating_profit"`, `"opm"`, `"net_profit"`, `"npm"`, `"eps"`, `"pe"`, `"pb"`, `"book_value"`, `"roce"`, `"roe"`, `"debt_to_equity"`, `"current_ratio"`, `"promoter_holding"`, `"fii_holding"`, `"dii_holding"`... --- # 7. 📄 My Portfolio Module ## What It Does (Plain English) Your one-stop view to see everything you own and have traded: - **Live Positions** — stocks you bought today (with real-time P&L) -

****Orders**** — all orders placed (pending, executed, cancelled) - ****Holdings**** — long-term stocks in your demat account - ****Paper Trading**** — practice trades without real money **### API Endpoints** (Already covered in Section 2.3) **### Frontend Components** | Component | Size | What It Shows | |-----|-----|-----| | `PortfolioView.tsx` | 45 KB | Tabs for Positions/Orders/Holdings/Paper, P&L summary | | `OrderPlacementModal.tsx` | 35 KB | Buy/sell form with SL, target, quantity calculator | --- # 8. 🚀 My Watchlist / Custom Portfolios **## What It Does** (Plain English) Create and save your own ****named portfolios**** based on screener formulas. For example: - ****"Buffett Quality Moat"***** → `roe > 15 AND opm > 12 AND debt\_to\_equity < 0.5` - ****"BSE-NSE Arbitrage"***** → `spread\_pct > 0.15 AND volume > 50000` The system ****dynamically evaluates**** these formulas against live market data, so the list of matching stocks changes in real-time. **### API Endpoints** | Endpoint | Method | Input | Output | |-----|-----|-----|-----| | `/api/v1/portfolios` | GET | None | `{portfolios[{id, name, formula, stock\_count, avg\_pe, avg\_roce, day\_change\_pct}], total}` | | `/api/v1/portfolios` | POST | `{name, formula, description?, tags?[]}` | `{success, portfolio, initial\_matches}` | | `/api/v1/portfolios/{id}` | GET | `?page=1&page\_size=50&sort\_by=&sort\_dir=` | `{portfolio, total\_stocks, stocks[], execution\_ms}` | | `/api/v1/portfolios/{id}` | PUT | `{name?, formula?, description?, tags?[]}` | `{success, portfolio}` | | `/api/v1/portfolios/{id}` | DELETE | portfolio_id | `{success, deleted\_id}` | **### Storage** ****File****: `backend/app/saved\_portfolios.json`

```
[
  {
    "id": "port-buffett-moat",
    "name": "Buffett Quality Moat",
    "formula": "roe > 15 AND opm > 12 AND debt_to_equity < 0.5 AND market_cap > 500",
    "description": "High ROE, strong margins, low debt",
    "tags": ["Quality", "Long Term", "Low Debt"],
    "created_at": 1726056000,
    "updated_at": 1726056000
  }
]
```

****Default seed portfolios****: 2 (Buffett Quality Moat + BSE-NSE Arbitrage) --- # 9. 🗄️ Data Vault (60-Day Tick Vault & Quant Matrix) **## What It Does** (Plain English) The Data Vault downloads and stores ****60 days of 1-minute candle data**** for every stock in the universe. This is approximately ****17 GB**** of granular price data that powers: - Historical backtesting - Quant analysis (26-parameter DNA scoring) - Win rate calculations - Pattern recognition --- **### 9.1 `historical\_batch\_engine.py`** > ****File****: `backend/app/engine/historical\_batch\_engine.py` · ****Lines****: 1,144 · ****Size****: ~49 KB **#### How the Sync Works**

1. User clicks "Start Sync" on Data Vault page
2. Engine queues all 5,092 stocks for download
3. Background thread downloads 60 days of 1-min candles per stock via Dhan API
4. Each stock's candles saved to intraday_history.db
5. Progress broadcasted to frontend (% complete, ETA, current symbol)
6. After download, 26-parameter quant scoring runs

Sync Modes | Mode | What It Downloads | Approximate Time | |-----|-----|-----| | `TOP\_100` | Top 100 liquid stocks | ~15 minutes | | `BSE` | All BSE equities | ~2 hours | | `FULL` | All BSE + NSE equities | ~6-8 hours | **#### API Endpoints** | Endpoint | Method | Input | Output | |-----|-----|-----|-----| | `/api/v1/historical-data/status` | GET | None | `{is\_syncing, progress\_pct, eta\_minutes, current\_symbol, total\_synced, total\_queued}` | | `/api/v1/historical-data/start` | POST | `{mode: "FULL"}` | `{success, message, total\_queued}` | | `/api/v1/historical-data/pause` | POST | None | `{success}` | | `/api/v1/historical-data/resume` | POST | None | `{success}` | | `/api/v1/historical-data/sync-stock/{symbol}` | POST | symbol | `{success, candle\_count, date\_range}` | | `/api/v1/historical-data/universe` | GET | `?page=&search=&status=&exchange=&target\_pct=&stop\_loss\_pct=` | `{stocks[], total}` | | `/api/v1/historical-data/candles/{symbol}` | GET | `?page=&page\_size=&date\_filter=&exchange=` | `{candles[], total, total\_pages}` | **#### 1-Minute Candle Storage** ****Database****: `intraday\_history.db` (~17 GB) ****Table****: `historical\_1min\_candles` | Column | Type | Description | |-----|-----|-----|

-----| | `symbol` | TEXT | Stock symbol (PK part 1) | | `timestamp` | INTEGER | Unix epoch seconds (PK part 2) | | `datetime\_str` | TEXT | Human-readable IST timestamp | | `open` | REAL | Opening price | | `high` | REAL | Highest price in that minute | | `low` | REAL | Lowest price in that minute | | `close` | REAL | Closing price | | `volume` | REAL | Volume traded in that minute | --- **### 9.2**

`quant\_copilot\_engine.py` — 26-Parameter Quant Scoring > ****File****: `backend/app/engine/quant\_copilot\_engine.py` · ****Lines****: 622 · ****Size****: ~36 KB Evaluates every stock against ****26 institutional-grade parameters**** across 4 categories: | Category | Parameters | Weight Range | |-----|:-----|:-----| | Structural & Trend (1-6) | Stage-2, EMA Slope, Higher-Low, 52W Proximity, Airspace, Liquidity | 3-4 | | Volatility & Action (7-12) | VCP Contraction, NR7, ATR Expansion, Wick Rejection, Close-to-High, Hurst | 4-5 | | Institutional Footprint (13-19) | RVOL, Volume Dry-Up, Delivery %, CVD, Ask Depth Chew, VWAP Hold, POC Shift | 3-5 | | Fundamentals & Sector (20-26) | RoCE, D/E, Quarterly Growth, Sector RS, Beta Decoupling, Pledge, Risk-Reward | 3-4 | --- **# 10.** 

Market Trends Module **## What It Does (Plain English)** Shows you the ****big picture of the market**** in one glance — is the market healthy? Which sectors are winning? Is momentum building or fading? Think of it as a ****market health dashboard****. **### 10.1**

`trends\_engine.py` > ****File****: `backend/app/engine/trends\_engine.py` · ****Lines****: 1,397 · ****Size****: ~70 KB Computes ****12 real-time metric cards**** across ****5 tabs****: **#### Tabs & Their Metrics** | Tab | Metrics Computed | What It Shows | |-----|-----|-----|

-----| | ****Market Breadth**** | Advance/Decline ratio, % stocks above 20/50/200 EMA, New Highs vs New Lows | Overall market health | | ****Price Action**** | Average change %, % stocks >2% up/down, 52W high count, gap-up/down count | Day's price action | | ****Volume**** | Total market volume, above-average volume count, delivery %, volume surge count | Institutional activity | | ****Technicals**** | RSI distribution, Supertrend bullish %, EMA crossover count, Bollinger squeeze | Technical signals | | ****Sector Rotation**** | 12 sectors ranked by day change, sector breadth, leading/lagging sectors | Sector flow | **#### 12 Sector Classifications** | # | Sector Name | Short Name | |---|-----|-----| | 1 | Banking, Financial Services & Insurance | Banking & Finance | | 2 | Information Technology | IT & Tech | | 3 | Automobiles & Auto Components | Auto & EV | | 4 | Power, Energy & CleanTech | Energy & Power | | 5 | Defense, Aerospace & Railways | Defense & Rail | | 6 | Pharmaceuticals & Healthcare | Pharma & Health | | 7 | Metals, Mining & Commodities | Metals & Mining | | 8 | Real Estate, Cement & Infrastructure | Realty & Infra | | 9 | Fast Moving Consumer Goods (FMCG) | FMCG & Retail | | 10 | Chemicals, Fertilizers & Agro | Chemicals & Agro | | 11 | Capital Goods, Engineering & Industrials | Capital Goods | | 12 | Media, Telecom & Consumer Services | Media & Telecom | **#### 30-Minute Rolling Pulse Snapshots** Every 30 minutes, the engine takes a ****pulse snapshot**** capturing the market's mood at that moment. These pulses are persisted to `recommendations.db` → `market\_day\_pulses` table and can be viewed historically. **#### API Endpoints** | Endpoint | Method | Input | Output | |-----|-----|-----|-----| | `/api/v1/universe/trends` | GET | `'?tab=breadth&exchange=ALL` | `{cards[{id, title, badge, signal, hero\_val, sub\_stat, insight}]}` | | `/api/v1/universe/trends/pulses` | GET | `'?date\_str=DD.MM.YYYY` | `{pulses[{slot\_time, sentiment, headline, story, sectors}]}` | --- **# 11.** 

Dalal Street News (Live Financial Intelligence) **## What It Does (Plain English)** Aggregates financial news from ****multiple RSS feeds**** across India's financial media landscape, classifies each article by sector, sentiment, and sensitivity, and lets you filter by any combination. **### 11.1** `news\_engine.py` > ****File****: `backend/app/engine/news\_engine.py` · ****Lines****: 919 · ****Size****: ~49 KB **#### News Classification System** Every article is automatically classified along 3 dimensions: | Dimension | Values | How It's Determined | |-----|-----|-----|

****Category**** | `ECONOMY`, `SECTOR`, `CORPORATE\_FILING`, `STOCKS` | Keyword matching in title/body | | ****Sentiment**** | `BULLISH`, `BEARISH`, `NEUTRAL` | Positive/negative word scoring | | ****Sensitivity**** | `HIGH`, `MODERATE`, `INFORMATIONAL` | Impact assessment keywords | **#### 31 Sector Keyword Matching** Each of the 31 sectors has a ****keyword dictionary**** for automatic sector tagging. Example: | Sector | Keywords | |-----|-----| | Banking & Financial Services | `bank, nbfc, rbi, repo, interest rate, npa, hdfc, sbi` | | Automobile & Electric Vehicles | `auto, vehicle, ev, car, maruti, tata motors, fada, siam` | | Information Technology | `it, software, infosys, tcs, wipro, hcltech, techm` | **#### Storage**: `news\_feed.db` Articles are stored in SQLite with fields: `id, title, summary, source, url, category, sector, symbol, sentiment, sensitivity, published\_at, ingested\_at` **#### API Endpoints** | Endpoint | Method | Input | Output | |-----|-----|-----|-----| | `/api/v1/news` | GET | `'?category=§or=&symbol=&sensitivity=&sentiment=&search=&limit=50&offset=0` | `{articles[], total, has\_more}` | | `/api/v1/news/match` | GET | `'?symbol=§or=` | `{article}` — best match for a stock | | `/api/v1/news/refresh` | POST | None | `{status, new\_articles\_ingested}` | --- **# 12.** 

Chart & Rules Studio (Dynamic Trigger Studio) **## What It Does (Plain English)** Create ****automated rules and triggers**** that fire when specific technical conditions are met. For example: "Alert me when RELIANCE's RSI drops below 30 AND price is above 200 EMA". **### 12.1** `rule\_engine.py` > ****File****: `backend/app/engine/rule\_engine.py` · ****Lines****: ~150 · ****Size****: ~6 KB Evaluates dynamic conditions against live stock data. **### 12.2** `strategy\_dsl.py` — Strategy Domain-Specific Language > ****File****: `backend/app/engine/strategy\_dsl.py` · ****Lines****: 92 · ****Size****: ~4 KB Defines the ****complete structure**** of a trading strategy using Pydantic models:

```

StrategyDefinition
├─ UniverseConfig      (which stocks to scan)
│  └─ base              "NIFTY_50" | "NIFTY_500" | "ALL_EQUITIES" | "CUSTOM"
│  └─ custom_symbols    ["RELIANCE", "TCS", ...]
│  └─ min_volume        100000
│  └─ min_price         10.0
├─ EntryRules          (when to buy)
│  └─ logic             "AND" | "OR"
│  └─ conditions[]      [{field, operator, value, timeframe}]
│  └─ time_filter_start "09:20"
│  └─ time_filter_end   "15:00"
├─ ExitRules           (when to sell)
│  └─ target_pct        3.0%
│  └─ stop_loss_pct     1.5%
│  └─ trailing_stop_pct 1.0%
│  └─ eod_square_off    true
├─ PositionSizing      (how much to invest)
│  └─ method            "RISK_BASED" | "FIXED_QTY" | "PERCENT_CAPITAL"
│  └─ capital_allocation 50000.0
└─ RiskLimits          (safety guardrails)
   └─ max_daily_loss    10000.0
   └─ max_open_positions 5
   └─ require_stop_loss true

```

Supported Condition Fields | Field | Description | Example | |-----|-----|-----| | `ltp` | Last Traded Price | `ltp > 1300` | | `rsi` | Relative Strength Index (14) | `rsi < 30` | | `ema\_20` | 20-period EMA | `ltp >= ema\_20` | | `ema\_50` | 50-period EMA | `ema\_20 > ema\_50` | | `ema\_200` | 200-period EMA | `ltp > ema\_200` | | `vwap` | Volume Weighted Average Price | `ltp CROSSES\_ABOVE vwap` | | `volume` | Current volume | `volume > 2.5 \* avg\_volume\_20d` | | `change\_pct` | Day's % change | `change\_pct > 2.0` | | `supertrend` | Supertrend indicator | `supertrend == "BUY"` | ### 12.3 Frontend Component

ChartRuleStudio.tsx — **242 KB** (the absolute largest component in the project) --- # 13. 🤖 Bot Studio (AI Co-Pilot & Strategy Builder) ## What It Does (Plain English) Build automated trading strategies by **chatting** with an AI assistant. Tell it what you want in plain English, and it generates a structured strategy that you can backtest and deploy: > **You**: "I want to buy Nifty 50 stocks when RSI drops below 30 and the price is near the 200 EMA support" > > **AI Co-Pilot**: "I've created an RSI Oversold Dip Buyer strategy with entry at RSI<30 + Price≥200 EMA, target 3.5%, stop loss 1.5%..." --- ### 13.1 `bot\_conversation.py` — AI Strategy Generator > **File**: `backend/app/engine/bot\_conversation.py` . **Lines**: 524 . **Size**: ~24 KB #### 6 Starter Templates | Template | Badge | Strategy Logic | |-----|-----|-----| | RSI Oversold Dip Buyer | Mean Reversion | RSI(14) <30 + Price≥200 EMA | | Intraday Volume Breakout | Momentum | VWAP Crossover + Volume>2.5x | | EMA Golden Cross | Trend | 20 EMA crosses above 50 EMA | | Supertrend Momentum | Trend Following | Supertrend BUY + Volume confirmation | | VWAP Reclaim | Intraday | Price reclaims VWAP + RSI>50 | | Tight Range Breakout | Volatility | NR7 + Volume expansion | ### 13.2

backtest_engine.py — Historical Simulation > **File**: `backend/app/engine/backtest\_engine.py` . **Lines**: 371 . **Size**: ~14 KB Runs a **deterministic historical simulation** on OHLCV bars: 1. Loads 120+ candles for target symbols 2. Evaluates entry conditions bar-by-bar 3. Simulates entries with exact position sizing 4. Applies exit rules (target, SL, trailing stop, EOD square-off) 5. **Deducts Indian statutory costs** (STT, brokerage, GST, stamp duty) 6. Produces equity curve, trade log, and performance metrics ####

Backtest Output Fields | Field | Type | Description | |-----|-----|-----| | `total\_trades` | int | Number of trades executed | | | `winning\_trades` | int | Trades that hit target | | | `losing\_trades` | int | Trades that hit stop loss | | | `win\_rate` | float | Win percentage | | | `total\_gross\_pnl` | float | P&L before costs | | | `total\_net\_pnl` | float | P&L after all costs | | | `total\_costs` | float | Sum of STT+brokerage+GST | | | `max\_drawdown\_pct` | float | Maximum peak-to-trough decline | | | `profit\_factor` | float | Gross profits ÷ Gross losses | | | `equity\_curve` | list | Time-series of portfolio value | | | `trades` | list | Detailed trade log with entry/exit | ### 13.3

cost_engine.py — Indian Regulatory Cost Calculator > **File**: `backend/app/engine/cost\_engine.py` . **Lines**: 325 . **Size**: ~13 KB Calculates **exact Indian market transaction costs**: | Cost Item | Type | Rate | |-----|-----|-----| | Brokerage | Flat per

- ### Password Rules - Minimum 6 characters, maximum 10 characters - At least 1 uppercase letter - At least 1 number - At least 1 special character (!@#\$%^&*) ### Storage - In-memory (`PENDING\_OTPS` dict, `USERS\_DB` dict) - No persistent database (resets on server restart) #### API Endpoints | Endpoint | Method | Input | Output | |-----|-----|-----|-----| | `/api/v1/auth/request-otp` | POST | `{email}` | `{success, message, dev\_otp}` | | `/api/v1/auth/verify-otp-password` | POST | `{email, otp, password, confirm\_password}` | `{success, email, token}` | | `/api/v1/alerts` | GET | None | `{alerts[], total}` — triggered rule alerts | | `/api/v1/alerts/clear` | POST | None | `{success}` | --- # 17. 🛠️ Admin Portal (Port 3001) ## What It Does (Plain English) A **separate, restricted dashboard** that only the admin (`somnathdey269@gmail.com`) can access. It provides: - **Strategy management** — promote Challenger → Champion versions - **Universe Inspector** — see every stock's solvency status and kill-switch reasons -

****Manual Stock Injector**** — force-add a recommendation - ****Quant Copilot**** — conversational AI analysis on 60-day data - ****EOD Reports**** — daily performance post-mortem - ****Solvency Gate**** — run fundamental safety checks across all stocks **### Admin Credentials** - ****Email****: `somnathdey269@gmail.com` - ****Password****: `Deevارش@190521` **### API Endpoints** | Endpoint | Method | Purpose | |-----|-----|-----| | `/api/v1/admin-portal/login` | POST | Admin authentication | | `/api/v1/admin-portal/candidates` | GET | All scored recommendation candidates | | `/api/v1/admin-portal/manual-inject` | POST | Force-add a recommendation | | `/api/v1/admin-portal/toggle-publish` | POST | Show/hide recommendation on public dashboard | | `/api/v1/admin-portal/strategies` | GET | All strategy versions with performance | | `/api/v1/admin-portal/strategies/activate` | POST | 1-Click Champion promotion | | `/api/v1/admin-portal/eod-reports` | GET | Daily post-mortem reports | | `/api/v1/admin-portal/settings` | GET/POST | Throttle & cadence settings | | `/api/v1/admin-portal/run-scan-now` | POST | Trigger immediate scan | | `/api/v1/admin-portal/universe-inspector` | GET | Full universe solvency status | | `/api/v1/admin-portal/solvency-status` | GET | Today's solvency cache | | `/api/v1/admin-portal/run-solvency-now` | POST | Force-run solvency gate | | `/api/v1/admin-portal/copilot/chat` | POST | Quant Copilot AI chat | | `/api/v1/admin-portal/copilot/stock/{symbol}` | GET | 26-parameter DNA profile | | `/api/v1/admin-portal/copilot/stocks-dna` | GET | Ranked stocks by confluence | --- # 18. ⚙️ **Settings & Configuration** **## What It Does (Plain English)** The Settings page is the ****control center**** for connecting all external services: **### Configuration Sections** | Section | What You Configure | Storage | |-----|-----| | ****Dhan Broker Connection**** | Client ID + Access Token | `.dhan_session.json`` | | **\*\*TOTP Auto-Login\*\*** | Client ID + PIN + TOTP Secret | `.dhan_totp_config.json`` | | ****xKiro AI Gateway**** | API Key + Model + Base URL | `.xkiro_config.json`` | | **\*\*Transaction Costs\*\*** | Brokerage, STT, GST rates | In-memory (CostEngine) | **### Frontend Component** **\*\*`AdminSettingsView.tsx`\*\*** — 31 KB --- # 19. 🌩️ **WebSocket Real-Time System** **## What It Does (Plain English)** A persistent WebSocket connection (``ws://127.0.0.1:8000/ws/terminal``) that sends live data to the frontend every 1.5 seconds without the browser needing to refresh. **### Data Flow**

DhanHQ MarketFeed WebSocket → `dhan_provider.stocks_cache` update

↓
`ConnectionManager.threadsafe_broadcast()`

↓
`/ws/terminal` → sends to all connected browsers

↓
Frontend receives MARKET_PULSE + TICK events

Message Types (Server → Client) | Type | Fields | Frequency | |-----|-----|-----| | ``MARKET_PULSE`` | ``indices[]`, top_gainers[]`, top_losers[]`, market_status`, is_market_open`, timestamp` | Every 1.5 seconds | | Stock Tick | `symbol`, ltp`, nse_ltp`, bse_ltp`, change`, change_pct`, volume`, day_high`, day_low` | On every price change | ### Client → Server Actions | Action | Payload | What It Does | |-----|-----|-----| | `SUBSCRIBE_STOCK` | {symbol: "RELIANCE"}` | Subscribe to real-time ticks for one stock | | `SUBSCRIBE_UNIVERSE` | {symbols: ["RELIANCE", "TCS", ...]}` | Subscribe to ticks for multiple stocks | --- # 20. 🌩️ Frontend Architecture ### App Router Pages | Route | Page | Component Loaded | |-----|-----|-----| | `/` | Root | Redirects to main dashboard | | `/recommendations` | AI Recommendations | `RecommendationDashboardView.tsx` | | `/vault` | Data Vault | `DataVaultView.tsx` | | `/portfolio` | My Portfolio | `PortfolioView.tsx` | | `/stocks` | Stocks Universe | `UniverseView.tsx` | | `/watchlist` | My Watchlist | `FinancialScreenerView.tsx` | | `/trends` | Market Trends | `TrendsDashboardView.tsx` | | `/news` | Dalal Street News | `NewsFeedView.tsx` | | `/chart` | Chart & Rules | `ChartRuleStudio.tsx` | | `/settings` | Settings | `AdminSettingsView.tsx` | ### Component Size Ranking | # | Component | Size | Purpose | |---|-----|-----|-----| | 1 | `ChartRuleStudio.tsx` | 242 KB | Interactive charting + rule builder | | 2 | `RecommendationDashboardView.tsx` | 172 KB | AI recommendations display | | 3 | `UniverseView.tsx` | 152 KB | Stock universe table | | 4 | `AdvancedScreenerModal.tsx` | 80 KB | Screener query builder | | 5 | `FinancialScreenerView.tsx` | 76 KB | Watchlist + screener | | 6 | `DataVaultView.tsx` | 46 KB | Historical data management | | 7 | `PortfolioView.tsx` | 45 KB | Portfolio positions/orders | | 8 | `OrderPlacementModal.tsx` | 35 KB | Buy/sell order form | | 9 | `AdminSettingsView.tsx` | 31 KB | Settings panel | | 10 | `StrategyScannerModal.tsx` | 30 KB | Strategy scanner | ### Type System **File**: `frontend/src/types/index.ts` — **853 lines, 19 KB** Key interfaces: - `StockQuote` — 32 fields for a single stock - `MarketIndex` — index value + change - `MarketTickerResponse` — WebSocket market pulse - `StockUniverseResponse` — paginated stock list - `TrendMetricCardData` — trend card data - `AuthUser` — user authentication state ### API Service Layer **File**: `frontend/src/services/api.ts` — **733 lines, 24 KB** All 60+ backend API calls organized by module: - Universe (5 functions) - Rules (5 functions) - Agents (6 functions) - Paper Trading (3 functions) - xKiro (5 functions) - Dhan (8 functions) - Auth`

(2 functions) - Alerts (2 functions) - Screener (7 functions) - Portfolios (5 functions) - Corporate Filings (2 functions) - Trends (2 functions) - Bots (7 functions) - News (2 functions) - Trade (7 functions) --- # 21. 📊 Database Architecture ## 7 SQLite Databases | # | Database | Size | Purpose | Location | |---|-----|-----|-----|-----| | 1 | `recommendations.db` | ~109 MB | Recommendations lifecycle, strategies, reports, pulses | `backend/app/engine/` | | 2 | `intraday\_history.db` | ~17 GB | 60-day 1-minute candle storage | `backend/app/engine/` | | 3 | `quant\_copilot.db` | ~19 MB | 26-parameter quant scoring | `backend/app/engine/` | | 4 | `news\_feed.db` | ~799 KB | News articles from RSS feeds | `backend/app/engine/` | | 5 | `corporate\_filings.db` | ~451 KB | BSE SEBI LODR filings | `backend/app/engine/` | | 6 | `bot\_studio.db` | ~20 KB | Saved bot strategies | `backend/` | | 7 | `stock\_screener.db` | Variable | Screener cache data | `backend/` | ### Key Tables Across All Databases #### `recommendations.db` — 8 Tables | Table | Fields Count | Purpose | |-----|:-----|:-----| | `strategy\_versions` | 12 | Champion/Challenger strategy versions | | `recommendations` | 30+ | Immutable recommendation audit log | | `rejected\_candidates` | 7 | "Why Not?" negative screening log | | `daily\_eod\_reports` | 14 | Post-mortem executive reports | | `admin\_settings` | 3 | Admin throttle & cadence config | | `market\_day\_pulses` | 15 | 30-minute rolling pulse snapshots | | `daily\_solvency\_cache` | 5+ | Daily fundamental safety cache | | `universe\_evaluations` | varies | Scored universe evaluation logs | #### `intraday\_history.db` — 2 Tables | Table | Fields Count | Purpose | |-----|:-----|:-----| | `historical\_1min\_candles` | 8 | Raw OHLCV 1-minute data | | `historical\_sync\_status` | 14+ | Sync progress + win rate metrics | #### `corporate\_filings.db` — 3 Tables | Table | Fields Count | Purpose | |-----|:-----|:-----| | `corporate\_results` | 22 | Quarterly/Yearly P&L statements | | `corporate\_balance\_sheet` | 18+ | Balance sheet data | | `company\_financials\_master` | 50+ | Consolidated financial master | --- # 22. 🚀 Startup & Background Services ## What Happens When the Server Starts When you run `uvicorn app.main:app`, these services start automatically:

```
Server Start (main.py → @app.on_event("startup"))
|
├─ 1. Dhan TOTP Service
|   └─ start_morning_scheduler() → 7:30 AM daily token renewal thread
|       └─ auto_refresh_if_needed() → checks if current token is expired
|
├─ 2. Corporate Filings Ingestion
|   └─ start_background_scheduler(interval_minutes=5)
|       → every 5 minutes, pulls BSE SEBI LODR announcements
|
├─ 3. Recommendation Engine
|   └─ start_background_workers()
|       → 30-minute scan cycles for new recommendations
|       → live price monitoring for entry/exit triggers
|
├─ 4. Trends Engine
|   └─ start_background_scheduler()
|       → 30-minute pulse snapshots
|       → real-time metric card calculations
|
└─ 5. WebSocket Manager
    └─ tick_listeners hook → broadcasts live ticks to connected browsers
```

--- # 23. 📖 Complete API Reference ## All Endpoints at a Glance #### Health | Method | Path | Description | |-----|-----|-----| | GET | `/api/v1/health` | Health check | #### Universe & Market Data | Method | Path | Description | |-----|-----|-----| | GET | `/api/v1/universe/stocks` | Paginated stock universe with filters | | GET | `/api/v1/universe/stocks/{symbol}` | Single stock data | | GET | `/api/v1/universe/stocks/{symbol}/chart` | OHLCV chart + technicals | | GET | `/api/v1/universe/stocks/{symbol}/details` | Company details + financials | | GET | `/api/v1/universe/indices` | Market indices (Nifty, Sensex) | | GET | `/api/v1/universe/ticker` | Market ticker data | | GET | `/api/v1/universe/sectors` | All 31 sector names | | GET | `/api/v1/universe/trends` | Market trend metric cards | | GET | `/api/v1/universe/trends/pulses` | 30-minute pulse snapshots | #### Recommendations | Method | Path | Description | |-----|-----|-----| | GET | `/api/v1/recommendations/metrics` | Today's performance scorecard | | GET |

``/api/v1/recommendations/sessions`` | Available session dates || GET | ``/api/v1/recommendations/active`` | Active recommendations || GET | ``/api/v1/recommendations/intraday/active`` | Intraday only || GET | ``/api/v1/recommendations/intraday/trimmed`` | Trimmed intraday trades || GET | ``/api/v1/recommendations/history`` | Historical completed trades || GET | ``/api/v1/recommendations/{id}/reason`` | Detailed evidence || POST | ``/api/v1/recommendations/scan`` | Trigger manual scan || POST | ``/api/v1/recommendations/reset`` | Wipe and regenerate | ### Live Trading (Dhan Broker) | Method | Path | Description | |-----|-----| | GET | ``/api/v1/trade/status`` | Broker connection status || POST | ``/api/v1/trade/settings`` | Update Dhan credentials || POST | ``/api/v1/trade/order`` | Place buy order || GET | ``/api/v1/trade/positions`` | Open positions + P&L || POST | ``/api/v1/trade/squareoff`` | Close a position || GET | ``/api/v1/trade/orders`` | All placed orders || POST | ``/api/v1/trade/order/cancel`` | Cancel pending order || GET | ``/api/v1/trade/holdings`` | Demat holdings | ### Screener & Portfolios | Method | Path | Description | |-----|-----| | GET | ``/api/v1/screener/metrics`` | Full metric catalog || GET | ``/api/v1/screener/apex-strategies`` | 32 preset strategies || POST | ``/api/v1/screener/evaluate`` | Evaluate screener query || GET | ``/api/v1/screener/custom-formulas`` | Saved custom formulas || POST | ``/api/v1/screener/custom-formulas`` | Create custom formula || DELETE | ``/api/v1/screener/custom-formulas/{id}`` | Delete formula || GET | ``/api/v1/portfolios`` | List saved portfolios || POST | ``/api/v1/portfolios`` | Create portfolio || GET | ``/api/v1/portfolios/{id}`` | Portfolio stocks || PUT | ``/api/v1/portfolios/{id}`` | Update portfolio || DELETE | ``/api/v1/portfolios/{id}`` | Delete portfolio | ### Bot Studio | Method | Path | Description | |-----|-----| | GET | ``/api/v1/bots/templates`` | Starter templates || POST | ``/api/v1/bots/chat`` | AI Co-Pilot conversation || POST | ``/api/v1/bots/backtest`` | Run historical backtest || GET | ``/api/v1/bots`` | List saved bots || POST | ``/api/v1/bots`` | Save bot || GET | ``/api/v1/bots/{id}`` | Get specific bot || DELETE | ``/api/v1/bots/{id}`` | Delete bot | ### News | Method | Path | Description | |-----|-----| | GET | ``/api/v1/news`` | Filtered news feed || GET | ``/api/v1/news/match`` | Best news match for a stock || POST | ``/api/v1/news/refresh`` | Refresh RSS feeds | ### Admin & Settings | Method | Path | Description | |-----|-----| | GET | ``/api/v1/admin/xkiro/status`` | xKiro connection status || POST | ``/api/v1/admin/xkiro/connect`` | Connect xKiro || POST | ``/api/v1/admin/xkiro/disconnect`` | Disconnect xKiro || GET | ``/api/v1/admin/xkiro/models`` | Available AI models || POST | ``/api/v1/admin/xkiro/test`` | Test xKiro connection || GET | ``/api/v1/admin/dhan/status`` | Dhan connection status || POST | ``/api/v1/admin/dhan/connect`` | Connect Dhan || POST | ``/api/v1/admin/dhan/disconnect`` | Disconnect Dhan || GET | ``/api/v1/admin/dhan/totp/status`` | TOTP auto-login status || POST | ``/api/v1/admin/dhan/totp/configure`` | Configure TOTP || POST | ``/api/v1/admin/dhan/totp/refresh-now`` | Force token refresh || GET | ``/api/v1/admin/cost-rules`` | Transaction cost rules || POST | ``/api/v1/admin/cost-rules/update`` | Update cost rule || POST | ``/api/v1/admin/cost-rules/add`` | Add custom cost || DELETE | ``/api/v1/admin/cost-rules/{id}`` | Delete cost rule | ### Authentication & Alerts | Method | Path | Description | |-----|-----| | POST | ``/api/v1/auth/request-otp`` | Request email OTP || POST | ``/api/v1/auth/verify-otp-password`` | Verify OTP + set password || GET | ``/api/v1/alerts`` | Triggered alerts log || POST | ``/api/v1/alerts/clear`` | Clear all alerts | ### Rules & Triggers | Method | Path | Description | |-----|-----| | GET | ``/api/v1/rules`` | List all rules || POST | ``/api/v1/rules`` | Create new rule || DELETE | ``/api/v1/rules/{id}`` | Delete a rule || PATCH | ``/api/v1/rules/{id}/toggle`` | Enable/disable rule || POST | ``/api/v1/rules/test`` | Test rule against live data | ### Corporate Filings | Method | Path | Description | |-----|-----| | GET | ``/api/v1/corporate-filings/status`` | Filings sync status || POST | ``/api/v1/corporate-filings/sync`` | Bulk sync filings || POST | ``/api/v1/corporate-filings/sync/{symbol}`` | Sync single stock | ### Historical Data | Method | Path | Description | |-----|-----| | GET | ``/api/v1/historical-data/status`` | Sync status & progress || POST | ``/api/v1/historical-data/start`` | Start 60-day download || POST | ``/api/v1/historical-data/pause`` | Pause download || POST | ``/api/v1/historical-data/resume`` | Resume download || POST | ``/api/v1/historical-data/sync-stock/{symbol}`` | Sync single stock || GET | ``/api/v1/historical-data/universe`` | Universe with win rates || GET | ``/api/v1/historical-data/candles/{symbol}`` | Raw 1-min candles | ### Admin Portal (Restricted) | Method | Path | Description | |-----|-----| | POST | ``/api/v1/admin-portal/login`` | Admin login || GET | ``/api/v1/admin-portal/candidates`` | Scored candidates || POST | ``/api/v1/admin-portal/manual-inject`` | Force-add recommendation || POST | ``/api/v1/admin-portal/toggle-publish`` | Show/hide recommendation || GET | ``/api/v1/admin-portal/strategies`` | Strategy versions || POST | ``/api/v1/admin-portal/strategies/activate`` | Promote strategy || GET | ``/api/v1/admin-portal/eod-reports`` | Daily reports || GET/POST | ``/api/v1/admin-portal/settings`` | Admin settings || POST | ``/api/v1/admin-portal/run-scan-now`` | Trigger scan || GET | ``/api/v1/admin-portal/universe-inspector`` | Universe solvency || GET | ``/api/v1/admin-portal/solvency-status`` | Solvency cache || POST | ``/api/v1/admin-portal/run-solvency-now`` | Run solvency gate || POST | ``/api/v1/admin-portal/copilot/chat`` | Quant Copilot chat || GET | ``/api/v1/admin-portal/copilot/stock/{symbol}`` | 26-param DNA || GET | ``/api/v1/admin-portal/copilot/stocks-dna`` | Ranked DNA list | **Total Endpoints: 72** --- # 24. 📖 Development Log > *Add entries here whenever something is built, fixed, or changed.* | Date | Module | Change | Author | |-----|-----| | Sep 2026 | All | Initial platform development and launch | Somnath | 12.09.2026 | Documentation | Created PLATFORM_BIBLE.md — comprehensive documentation | Somnath | --- # 25. 💡 Discussed But Not Yet Built > *Capture ideas, features, and conversations that haven't been implemented yet.* | # | Feature Idea | Module | Status | Notes | |---|-----|-----|-----| | 1 | *Add entries as discussions happen* | — | 🗨️ DISCUSSED | — | --- # 26. 🗺️ Roadmap & Backlog > *Prioritized list of future enhancements.*

| Priority | Feature | Estimated Effort | Status | |-----|-----|:-----:|:-----:| |  HIGH | *To be added* | — | 

PLANNED | --- # 27.  Known Issues & Gotchas > *Things that work but have edge cases, workarounds, or known limitations.* | # | Issue | Module | Workaround | Severity | |---|-----|-----|-----|:-----:| | 1 | DhanHQ MarketFeed WebSocket occasionally disconnects during market hours | Dhan Provider | Auto-reconnect logic + patched `\_is\_ws\_closed()` |  MEDIUM | | 2 | Auth is in-memory only — all users lost on server restart | Auth | Use for dev only; persistent DB needed for production |  LOW | | 3 | `intraday\_history.db` grows to ~17 GB on full sync | Data Vault | Consider periodic pruning or archival strategy |  MEDIUM | | 4 | DhanHQ SDK `DhanHTTP.\_parse\_response` swallows error details | Dhan Provider | Monkey-patched at startup to extract real error messages |  RESOLVED | --- # 28.  Design Decisions > *Why certain technical choices were made.* | Decision | Rationale | |-----|-----|-----| | ****SQLite over PostgreSQL**** | Zero-config, single-file databases, embedded with Python. Perfect for single-user/small-team deployment. Can migrate to Postgres later if scaling needed. | | ****DhanHQ as primary broker**** | Full-featured Indian broker API with WebSocket market feed, historical data API, and order execution. TOTP auto-login eliminates daily manual token hassle. | | ****xKiro as AI gateway**** | OpenAI-compatible API that provides access to multiple model providers (Claude, GPT, etc.) through a single API key. Avoids vendor lock-in. | | ****Next.js 16 + React 19**** | Latest stable versions with App Router, Server Components, and superior dev experience. | | ****In-memory stock cache**** | 5,092 stocks × ~50 fields each fits comfortably in memory (<100 MB). Eliminates DB read latency for real-time price queries. | | ****AST-based screener**** | Python's `ast` module provides safe expression evaluation without `eval()`. Prevents code injection while allowing complex mathematical queries. | | ****18-parameter scoring**** | Multi-factor approach prevents single-indicator false signals. Minimum 10/18 threshold ensures comprehensive evidence before recommending. | | ****1-minute candle granularity**** | Finest granularity available from Dhan API. Enables precise backtest simulation and microstructure analysis that 15m/1D candles cannot provide. | --- >  ****This is a living document.**** Update it whenever you build, fix, discuss, or plan anything for the platform. > Last generated: 12 September 2026 · Apex Indian Equities Terminal v2.0.0]]>